

Review

Edge Computing and Its Application in Robotics: A Survey

Nazish Tahir * and Ramviyas Parasuraman *

School of Computing, University of Georgia, Athens, GA 30602, USA

* Correspondence: nazish.tahir@uga.edu (N.T.); ramviyas@uga.edu (R.P.)

Abstract

The edge computing paradigm has gained prominence in both academic and industry circles in recent years. When edge computing facilities and services are implemented in robotics, they become a key enabler in the deployment of artificial intelligence applications to robots. Time-sensitive robotics applications benefit from the reduced latency, mobility, and location awareness provided by the edge computing paradigm, which enables real-time data processing and intelligence at the network's edge. While the advantages of integrating edge computing into robotics are numerous, there has been no recent survey that comprehensively examines these benefits. This paper aims to bridge that gap by highlighting important work in the domain of edge robotics, examining recent advancements, and offering deeper insight into the challenges and motivations behind both current and emerging solutions. In particular, this article provides a comprehensive evaluation of recent developments in edge robotics, with an emphasis on fundamental applications, providing in-depth analysis of the key motivations, challenges, and future directions in this rapidly evolving domain. It also explores the importance of edge computing in real-world robotics scenarios where rapid response times are critical. Finally, the paper outlines various open research challenges in the field of edge robotics.

Keywords: edge computing; robotics; wireless robotic networks

Academic Editors: Dionisis Kandris,
Eleftherios Anastasiadis and Purav
Shah

Received: 14 April 2025

Revised: 12 June 2025

Accepted: 16 June 2025

Published: 23 June 2025

Citation: Tahir, N.; Parasuraman, R.
Edge Computing and Its Application
in Robotics: A Survey. *J. Sens.*

Actuator Netw. **2025**, *14*, 65. <https://doi.org/10.3390/jsan14040065>

Copyright: © 2025 by the authors.
Licensee MDPI, Basel, Switzerland.
This article is an open access article
distributed under the terms and
conditions of the Creative Commons
Attribution (CC BY) license
(<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

Edge computing is emerging as a new research hotspot in the computing landscape [1–3]. It is a novel computing paradigm that performs computation at the network's edge. Edge computing differs from traditional cloud computing concepts in that it allows computation to take place closer to the source of data. It provides the same services as cloud servers, but in close proximity, which facilitates quicker processing and hence faster execution time [4]. Zha et al. [5] defined the concept of edge computing: “Edge computing is a computing model that unifies resources that are close to the user in geographical distance or network distance to provide computing, storage, and network services for applications”.

Edge computing, in essence, enables the migration of the cloud's network, computing, and storage capabilities to the network's edge and the provision of intelligent services at the edge to meet the high computational demands of the data optimization and application intelligence of the fast-paced IT industry. It facilitates the execution of application intelligence on the network with low latency and high bandwidth. It expands the cloud network by enhancing computation, storage, and resource capabilities at the network's edge, close to the source of data, with the goal of meeting critical needs such as real-time servicing, application intelligence, security, and privacy, as well as the network's criteria for low latency and high bandwidth [6].

A brief taxonomy of existing computational domains related to cloud computing concepts is presented in Figure 1.

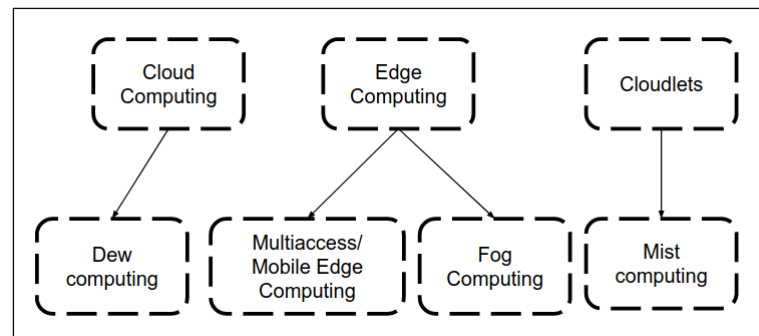


Figure 1. Taxonomy of existing computational domains. Note: Fog computing is often used interchangeably with edge computing in literature and is classified accordingly here.

The promising features of edge computing are *fast data processing and analysis*, leading to quicker response time and speed. Since edge devices are closer to the data source, they eliminate the need for intermediate data pre-processing and enable a faster transmission process. One of the most crucial aspects of edge computing is its *proximity to users*, which allows for the provision of better data-intelligent services due to increased digital data transfer over networks across shorter distances, *minimizing latency*. Furthermore, unlike cloud computation, which entails total offloading of data for processing and a centralized processing approach, it allows for *distributed and parallel processing*, saving time and ensuring data security [7]. It offers *low cost, low bandwidth cost, and energy consumption* [8] which makes it an ideal alternative to cloud computing.

Outsourcing all the required tasks to the cloud data center might place a significant strain on the network and trigger network congestion and network delays, particularly for large data volume exchange. Because of such network delays, the cloud is inadequate for addressing real-time, low-latency, and high-quality-of-service applications. The use of edge reduces not only the stress on network bandwidth but also the entire energy consumption of local devices associated with data transmission and processing, thereby increasing computing efficiency significantly.

1.1. Applications of Edge Computing in Robotics

In recent years, there has been a significant emphasis in the industry on developing smart Cyber-Physical Systems (CPSs) that are deployed in healthcare, transportation, and agriculture sectors to conduct complex engineering processes with less direct human intervention and improved cost and performance. Robots are a crucial component of this CPSs system, which rely on automated tasks in the sense–compute–act cycle [9].

Although a robot is equipped with sufficient computing resources to perform small-scale operations, it is heavily reliant on remote resources to perform large-scale computations, as the robot's onboard computational capacity may be insufficient to perform real-time data processing on large chunks of data [10]. Offloading enables robot operators to access data from anywhere at any time by providing global storage and processing capabilities. Since the concept of CPS is heavily reliant on the timely delivery of suitable data with the node placement to the relevant computing entities, robots that move resource-intensive tasks to remote resources require efficient solutions with minimal latency.

To accomplish this objective, the concept of cloud computing has been applied to multirobot systems, known as cloud robotics [11,12]. In this paradigm, the cloud provides computational resources such as virtual machines or containers, as well as resources from both local and remote data centers, allowing scalable and vast data processing [13]. However, because cloud servers are located in remote places, most of which are multi-hop distance away, connection time and data transfer latency might be categorized as a key drawback of cloud robotics, particularly for time-sensitive applications.

Edge computing offers a promising solution to this limitation by providing cloud-like infrastructure at the network's edge, allowing mobile robots to access computational and storage resources. Edge computing capitalizes on the necessity for real-time task execution in close proximity to these smart systems, such as image processing and path planning (Figure 2). In the context of CPS, edge computing enables robots to conduct a variety of robotic applications with remote support. It provides abstraction to classifying robotic activities based on computing needs and offers distributed resources to analyze large amounts of data with minimal communication latency [14]. It also boasts the robots' competency in knowledge exchange by leveraging local edge devices rather than distant cloud servers. Moreover, it contributes to lowering the bandwidth required for data transfer to the cloud.

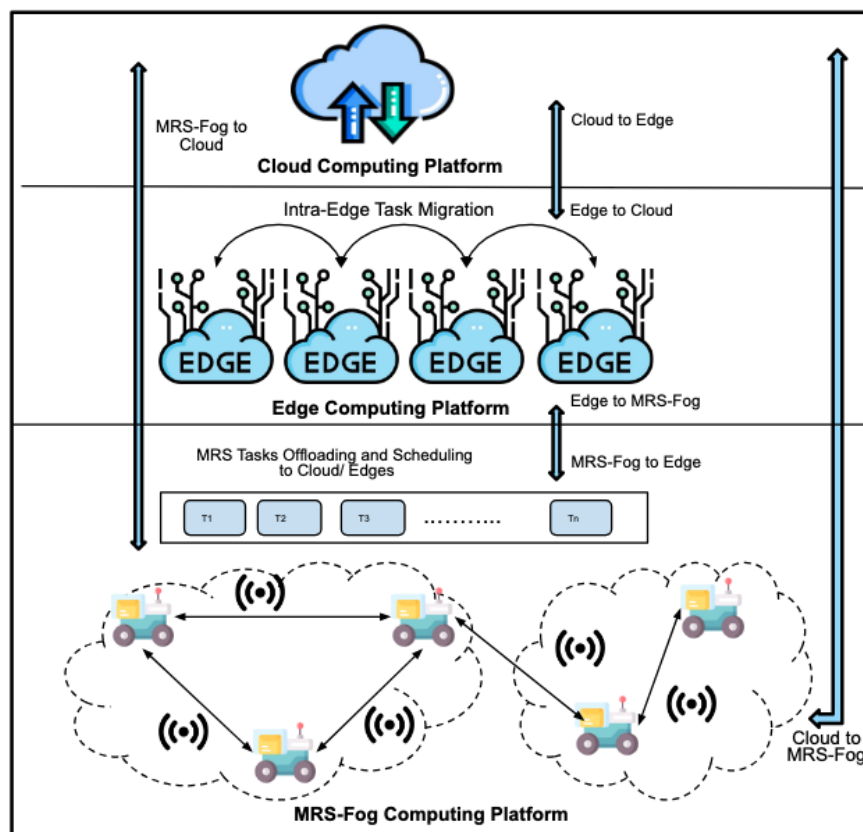


Figure 2. Overview of robot applications supported by edge, cloud, and fog (multi-robot collaboration) infrastructures.

1.2. Survey Scope

Several studies published in reputable journals have surveyed the use of edge computing in robotics, with a primary focus on specific experimental prototypes and system architectures. These works have been instrumental in highlighting the limitations of cloud computing in addressing the operational challenges of robotic systems. For example, M. Groshev et al. [15] provide a valuable contribution by analyzing the current landscape

of edge computing in robotics and experimentally evaluating an end-to-end robotic system based on existing solutions. Their study identifies key challenges related to system operation and network limitations, as well as the need for improved orchestration mechanisms. However, while their work offers important insights into specific applications and experimental results, it lacks a comprehensive review of the broader literature and does not provide a detailed classification of existing solutions in edge robotics.

Other works, such as [16], take a broader societal and cross-domain perspective, focusing on taxonomy, standardization, and human–machine interaction, rather than specifically addressing edge computing or architectural evaluations in multi-robot systems. While they touch on certain aspects relevant to edge robotics, they do not provide a comprehensive or focused study of the field.

There have been other efforts in comprehensively designing a survey study of edge computing in robotics in specific fields like agriculture [17], Industry 5.0 with one section focusing on industrial robots [18], and healthcare service robotics [19].

Our work distinguishes itself by presenting a comprehensive survey of edge robotics, encompassing a structured classification based on application domains, comparative analysis, and an in-depth discussion of open research challenges. We begin by establishing the necessary background to distinguish among various distributed computing paradigms, positioning edge robotics within this broader context. The survey then investigates the core characteristics of edge robotics, reviews state-of-the-art applications across a range of robotic tasks, and concludes with a discussion of key challenges and potential future directions. Thus, the main contributions of this paper are summarized as follows:

- A classification of the diverse computing paradigms relevant to edge robotics;
- A survey of the current landscape and core aspects of edge robotics;
- An identification and discussion of key research challenges and future directions.

We conduct an in-depth literature review using three major academic databases—IEEE Xplore, ScienceDirect, and the ACM Digital Library—to investigate current and emerging research on edge robotics. By employing search terms such as edge computing in robotics, multi-robot edge systems, multi-edge servers in multirobot systems, and edge robotics, we identify a substantial body of relevant publications spanning from 2015 to 2025. This review provides valuable insights into the evolution of edge robotics, highlighting both historical developments and the current state of the technology.

1.3. Article Organization

The remainder of the article is organized as follows: Section 2 introduces the fundamental principles of cloud robotics, edge robotics, fog robotics, MEC, and other similar topics. Section 3 delves into the properties of edge robotics as well as the reason for its adoption. Section 4 investigates the state of the art in edge robotics and presents a comparative evaluation of research work that shows the benefits and drawbacks of existing frameworks. Section 5 provides a focused review of experimental studies in edge-assisted robotics, highlighting key limitations in their setup, evaluation scope, and unresolved research gaps. Section 6 delves into the open research challenges of edge robotics implementation. Section 7 brings the paper to a close. For a better understanding, the organization of this article is also depicted in Figure 3.

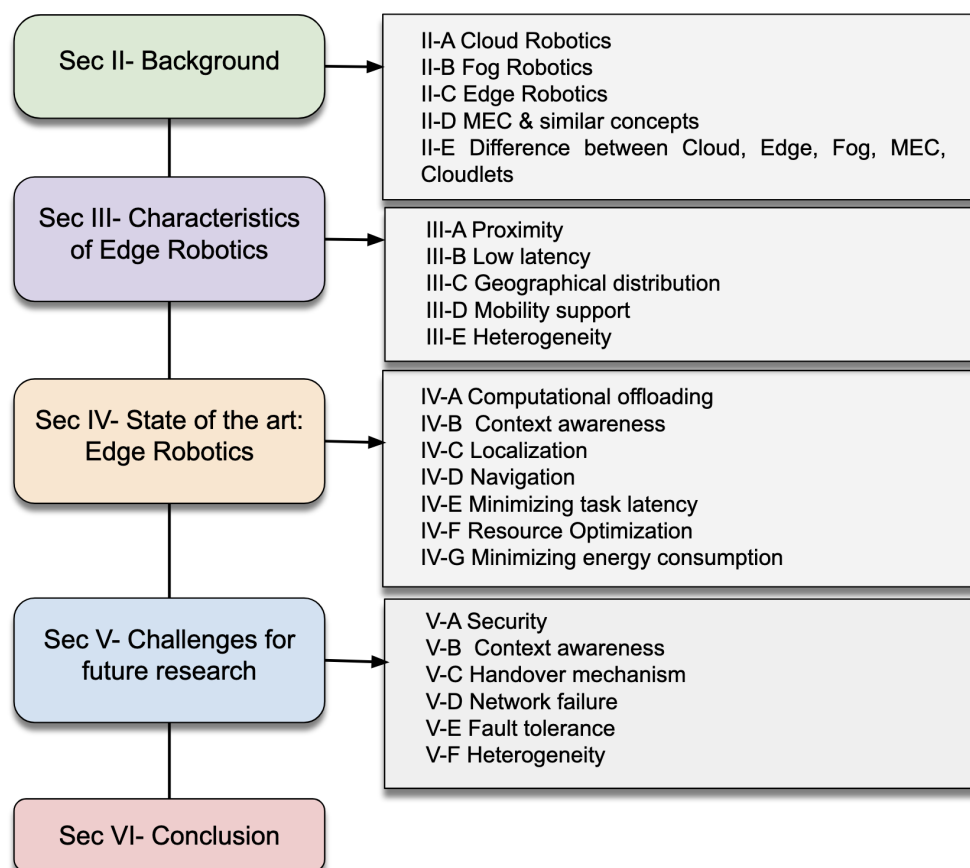


Figure 3. Table of contents and article organization.

2. Background

2.1. Cloud Robotics

The term “cloud robotics” was first coined by Kuffner in 2010 [20]. It represents any robot or automation system that utilizes cloud infrastructure for data or code execution—that is a system in which all sensing, computing, and memory are not confined into a single independent unit. Cloud-enabled robots can perform complex tasks considerably more efficiently by leveraging massively parallel computation, grid computing capabilities, and enhanced storage capacity [12].

Cloud computing offers dynamic and on-demand computing resources with centralized storage that can be accessed from anywhere at any time enabling remote robot control and monitoring [11]. Cloud robotics fosters collaboration among geographically dispersed robots by providing access to extensive datasets, including global maps for localization, object models for manipulation, high-computational machine learning techniques, and shared code repositories [21]. There are several cloud robotics system architectures available, like DAVinCi [22], which is a software framework based on a Hadoop cluster with Robot Operating System (ROS) as a messaging framework, which can be efficiently utilized in applications like FastSLAM [23].

Another popular open-source cloud robotics platform called Rapyuta [24], driven by cloud robotics, enables the robots to offload computational tasks while offering them access to the RoboEarth knowledge database. Similarly, C²TAM [25] is a cloud robotics framework especially designed for cooperative tracking and mapping to solve SLAM problems. Amazon’s AWS RobotMaker [26] has recently emerged as a cloud-based simulation tool that allows developers to perform simulations without the need to manage underlying infrastructure.

Cloud robotics applications span a wide range of domains including localization, navigation, grasping and manipulation, perception and computer vision, service robots, social and medical robots, manufacturing, and human–robot interaction. The current research problems in cloud robotics include establishing effective resource and task allocation schemes via the cloud, minimizing communication delays associated with network and data processing, and enhancing privacy and security for cloud-based robotic applications [27].

2.2. Fog Robotics

In order to overcome the challenges associated with cloud robotics, Cisco introduced the concept of fog computing [28]. Based on the same concept as edge robotics, fog robotics (FR) can be defined as an architecture that deploys small platforms placed at the network edge to offer storage, network functions, and control with decentralized computing closer to robots. The fog robotics architecture also involves a cloud system in addition to a Fog Robot Server (FRS) [29]. Any multi-robot system (MRS) requests the fog layer for data access and, upon availability, can efficiently utilize the resources available at the fog without querying the cloud.

The benefits of using fog robotics for robotic applications are closely similar to the ones offered by edge robotics, i.e., offering remote computational offloading, remote monitoring and control of the robots, security services, communication services, storage and caching services, offering global synchronization and collaboration within MRSs, mobility support, navigation, and scalability. Fog robotics helps assist household robotics by allowing models trained on the cloud to be regularly pushed to a smart home gateway rather than directly downloaded to individual robots. In other words, it offers a local model repository cache to robots so that they may adapt models or run a shared inference service for local robots to support those with restricted onboard resources.

Different architectures are supported by the fog-based MRS system [30]. They have been classified as *Nanoscale MRS* designed to efficiently service nanoscale robots, *Body Area MRS* which comprises an MRS system deployed on a single mobile body like a human or a mobile structure like wearable robots that require wireless connectivity with a single fog node, and *Local Area MRS* that is deployed in a limited geographic area, allowing robots to connect to each other through a wireless connected fog layer for better performance, reliability, and scalability.

There are various issues associated with fog robotics that may be investigated in the literature. The data stored at the fog layer should be maintained depending on the local movement of the robots; therefore, effective data maintenance and storage are required. There is a handover delay during data transfer between robots and fog servers that should be examined and regulated. Robust security protocols must be in place to prevent malicious robot users from accessing fog-based systems. Additionally, the status of a robot's battery must be taken into account when processing tasks at the fog so that the robot can complete the entire task execution, including data transfer from the fog layer.

2.3. Edge Robotics

Edge robotics aims to address the issues associated with cloud and fog robotics by extending the storage and processing capacity of a large number of robots linked to a network, offering an intermediary layer between the robots and the cloud. Unlike fog robotics, edge robotics can handle computational data distributively at the edge devices without the need to offload it to the cloud. Fog computing nodes (FCNs) handle the task by computing and storing the data from mobile robots locally before handover to the cloud while edge nodes may solve the data query solely on the edge layer. The existence of network edges reduces the computational load of mobile robots by allowing them to

handle some of the requests addressed to the cloud locally—those that do not require cloud intervention. This minimizes delay in resolving queries and enables real-time handling of a either a subset of requests or the entire dataset in parallel.

2.4. MEC and Similar Concepts

Mist Computing is another term that originates from the traditional edge computing domain. Mist computing involves the use of complementary embedded peripheral devices to preprocess or filter sensory input before sending it to the fog or cloud level. Complex sensors and actuators, such as cameras, 3D scanners, and laser range finders, are equipped with a microcontroller unit or field-programmable gate array that may execute certain functions; nevertheless, their processing capabilities are restricted. Mist computing is the initial computing point in the robot–fog–cloud continuum, allowing compute, storage, and networking capabilities from the fog to the objects [31].

Another computing paradigm to emerge from edge computing is *Mobile Edge Computing (MEC)* standardized by European Telecommunications Standards Institute (ETSI) [32]. As of 2017, the ETSI renamed MEC to “Multi-Access Edge Computing” to reflect growing interest by non-cellular operators [33]. MEC is an edge computing technology that brings processing and storage capability to the network’s edge within the Radio Access Network (RAN) to minimize latency and increase context awareness. MEC servers function in tandem with radio network controllers or macro base stations. The servers execute many instances of the MEC host, which may compute and store data across a virtualized interface. MEC reduces the computational and energy consumption of mobile devices by bringing the resources closer to the base stations, thus enabling one-hop services, minimizing the latency associated with data transmission.

Although originally developed to cater to the computational incapacity of cellular mobile devices like smartphones, MEC has been extended to the CPS systems. Since MEC falls under the broader umbrella of edge computing, we consider robotic applications using MEC to be part of the edge robotics paradigm in our survey of state-of-the-art approaches. MEC provides a logical solution to enhance robot collaboration because it can not only offer efficient communication among robots but also efficiently distribute tasks. The authors in [8] proposed a computational resource allocation strategy between mobile users and MEC servers to reduce task implementation delay and energy usage. Many such works utilizing MEC are discussed in Section 4.

Further extending the particularities of MEC, *Cloudlets*, as envisioned by the researchers [34], are small clusters or data centers capable of processing and storage that are located closer to mobile devices. Often referred to as mini-clouds, cloudlets address the drawbacks associated with traditional cloud computing platforms. The primary purpose of a cloudlet is to improve the interactive performance of mobile apps, particularly those with high end-to-end latency and jitter requirements. To meet the need for minimal delay, the proximity of cloudlets allows servers to deliver highly responsive cloud services to mobile nodes. The authors of [35] target the energy consumption and service delay reduction in multirobot systems by distributing resources in a cloudlet-based emergency management service within a smart factory.

Another concept to emerge from the umbrella of remote computational platforms is Dew computing, which is beyond the scope of our survey.

2.5. Difference Between Cloud, Edge, Fog, MEC, Cloudlets

The primary distinction between the aforementioned computing paradigms is that although cloud computing provides centralized processing, other computing paradigms, such as edge, fog, mist, and cloudlets, are built for distributed processing. Cloud comput-

ing allows for high scalability and cost-effective storage, but edge and other computing architectures may necessitate smaller, specialized processing equipment. Edge computing provides lower latency and response times, reduced power consumption and bandwidth costs, and enhanced security compared to the cloud.

The terms “edge” and “fog” are sometimes used interchangeably due to conceptual similarities; however, there are important distinctions. Both permit data transmission from robots to the cloud, but edge computing allows complete, localized data processing at the edge, removing the need for a cloud entirely. In contrast, fog computing acts as a mediator between the edge and the cloud for data pre-processing before cloud upload. Edge robotics may entail connecting to the robot’s sensors and controllers to deliver large amounts of data to the cloud, or alternatively, these data may be processed entirely at the network edge. The fog, on the other hand, is a computational layer that may accept data from the edge layer and analyze them before sending them to the cloud. Fog computing eliminates unnecessary data to reduce clutter in the cloud, offering lower latency and better efficiency of data traffic. However, one downside of fog computing is that it cannot fully replace edge computing, and the latter can be implemented without the additional infrastructure overhead that fog requires. Edge computing, on the other hand, has fewer peripheral layers than fog and thus offers less scalability. It cannot provide resource pooling and interoperability to the extent that fog can. Nonetheless, it relies on multiple links for data transport, which is a significant disadvantage of fog computing. In short, both edge and fog computing in robotics facilitate low-latency, secure, and efficient processing close to robotic platforms, enabling real-time tasks such as object detection and control. Edge computing primarily involves static infrastructure for computation offloading [36], whereas fog computing builds upon this by incorporating mobile and resource-constrained devices, offering enhanced flexibility, security, and context awareness across the cloud-to-edge spectrum [37]. Table 1 provides a side-by-side comparison of edge and fog computing in the context of robotics. For a deeper exploration of the challenges involved in integrating fog computing into robotic systems, we refer readers to [38,39].

Table 1. Comparison Between Fog and Edge Robotics.

Aspect	Fog Robotics	Edge Robotics
Scope	Broader, covers intermediate layers between cloud and robots (e.g., gateways, local servers)	Narrower, focused on computing at or near the robot or sensor
Location	Distributed across network nodes between cloud and robots	Located directly at or on the robots or edge devices
Control and Coordination	Managed across multiple network nodes, e.g., warehouses or local hubs	Mostly local control near the robot or sensor
Latency and Bandwidth	Reduces latency and bandwidth usage by intermediate processing	Minimizes latency by ultra-local processing near the robot
Example Use Case	Warehouse robots coordinating via local servers with occasional cloud access [40]	Robots processing sensor data on nearby embedded devices for real-time tasks [41]
Administrative Domain	Can span multiple trusted domains with policies on data flow	Typically operates within a single domain such as a smart home or manufacturer

For fog, MEC, and cloudlet paradigms, end devices and edge servers all install operating systems and application-specific software. Edge servers or the cloud serve as platforms for end devices to offload computation-intensive tasks to ensure timely data processing and energy efficiency. However, for fog and cloudlets, complete offloading is not feasible, as they are generally capable of executing only the necessary code rather than the entire program. Cloudlets mainly use virtual machines (VMs) for virtualization, while MEC and fog can also leverage containers for lighter-weight and faster deployment [42].

3. Key Characteristics of Edge Robotics

Edge robotics possesses several characteristics that mirror those of edge computing. The following are the defining features of edge computing that make it beneficial for deployment in robotics applications.

3.1. Proximity

Edge computing makes remote computational resources and services available to the robots in close proximity, which can improve their performance for time-sensitive applications. The availability of computing resources and services directly at the edge of the network enables robots to use context-aware network information to make offloading and service-based decisions. Robots can be made more intelligent by utilizing the remotely available services to realize the concept of “remote brain” for the efficient execution of logical algorithms.

3.2. Low Latency

Edge computing brings processing resources and services closer to robots, reducing latency in service access. Edge computing’s low latency enables robots to run resource-hungry and delay-sensitive applications on resource-rich edge devices (e.g., router, access point, base station, or dedicated server) for faster execution of algorithms like SLAM, navigation, object detection, gesture recognition, etc.

3.3. Geographical Distribution

Unlike cloud services, which are located far from local robots, edge computing brings the compute infrastructure closer by allowing similar service provisioning as the cloud. Their dense geographical distribution makes them an excellent contender for computational offloading in mobile robots, as it allows better fault tolerance or resilience to network loss through faster backups and recovery.

3.4. Mobility Support

Edge computing offers mobility by utilizing protocols like the Locator/ID Separation Protocol (LISP) to interact directly with mobile devices or robots. The LISP protocol separates host identity from location identity and creates a distributed directory system. The basic idea that enables mobility support in edge computing is this decoupling, allowing mobile robots to access services closer to their current location, thereby enhancing performance.

3.5. Heterogeneity

The existence of various platforms, architectures, infrastructures, processing, and communication technologies employed by edge computing elements is referred to as heterogeneity in edge computing (end devices, edge servers, and networks). Edge devices can also be customized, adding further layers of heterogeneity, including software, hardware and technological variances, APIs, custom-built policies, and platforms. In addition, network-level heterogeneity may emerge, leading to a variety of communication technologies that influence the delivery of edge services to the mobile robots.

4. State of the Art: Edge Robotics

This section critically analyzes the literature on edge robotics paradigms, including Mobile Edge Computing (MEC) in robotics, with a focus on their objectives and outcomes.

4.1. Computational Offloading

By employing a multi-tier cloud and edge computing architecture to dynamically offload computationally expensive parts of the algorithm, Dey et al. [43] demonstrated the usefulness of an edge computing platform for SLAM performance optimization. The edge server decides on offloading depending on compute and communication load, as well as energy utilization, with the goal of reducing execution time. To solve the SLAM challenge, the authors used a modified particle filter technique in conjunction with a dynamic offloading decision-making mechanism, evaluating the strategy through experiments conducted on VMs. The results revealed that the proposed dynamic offloading strategy consistently outperformed the static offloading, significantly reducing the mean execution time. However, no real-world testing was performed, where the communication layer could become a bottleneck due to high data transmission between robots and edge gateways.

Dechouniotis et al. [44] presented a uniquely developed architectural testbed, NET-MODE, based on the edge computing domain designed to manage heterogeneous computing, network resources, and mobile robots for complex smart manufacturing scenarios. An edge-assisted SLAM application is deployed and tested using the suggested architecture which advanced previous efforts through experiments with SLAM algorithm variations based on map update interval value (in seconds) implemented on local and edge devices provided a comparison of CPU use of the *gmapping* method on the two resources, thus addressing earlier lack of real-environment testing. The experimental findings showed considerable acceleration in the performance of compute-intensive workloads when edge infrastructure was used.

Li et al. [45] suggested an intelligent control approach to meet the computing power needs of AI high-precision algorithms, particularly when a large number of robots create vast data in real time. The authors evaluated and compared existing working techniques for robotic inspection of power grid stations and introduced an intelligent control strategy for live robot operation based on cloud and edge computing infrastructure. They analyzed the results of experiments on live working robots, which show that by using edge computing to filter data for the cloud platform, their proposed control method can meet the computational power requirements of AI deep learning, neural network coordinated control, self-learning control models, and so on, and it can be extended to multiple large numbers of robots working simultaneously generating massive data. Their edge module comprises several heterogeneous components equipped with GPU, NPU, TPU, APU, and FPGA capabilities. It processes sensor inputs, generates operation instructions, filters status information and algorithm parameters, and receives updates and training from the cloud. Thus, their proposed work moves beyond SLAM to support scalable, real-time AI computation across robot fleets, enabled by a diverse hardware edge ecosystem.

A new perspective of integrating UAVs for adaptive coverage was proposed by Wang et al. [46], thus shifting focus to a collaborative mobility-aware strategy. The proposed collaborative task offloading strategy uses improved genetic algorithms in Mobile Edge Computing (MEC) introducing the unmanned aerial vehicle (UAV) cluster addressing the limitations of fixed base stations in geographically constrained areas. The authors put forward a collaborative task offloading model to offload tasks to UAVs or a base station. An objective function is put forward to jointly minimize task latency and energy consumption, and the genetic algorithm solves the resource optimization problem by proposing an optimal collaborative task offloading strategy. Although simulation results provide supportive evidence for performance gains, the model's applicability under dynamic and unpredictable conditions in real-world scenarios remains unanswered.

Recent work by Tahir and Parasuraman [47] introduced a utility-aware dynamic task offloading strategy for multi-edge robotic systems, aiming to minimize task latency

and enhance resource utilization for sequential multi-robot tasks. In another study [48], the authors proposed a decentralized, edge-enabled scheduling strategy for collaborative multi-robot systems, optimizing resource use and task performance, and demonstrating improvements in latency, throughput, and frame rates compared to traditional edge-based methods. This work provides a distinctive perspective on multi-robot coordination and utility-driven decision-making under edge environments, moving from individual offloading to team-based offloading decisions.

Baruffa et al. [49] added a communication-centric approach to the existing literature by proposing a testbed architecture that combines cloud-edge computing and multi-RAT (Radio Access Technology) networks for mobile robotic applications. Their proposed work enables computational offloading for AI-driven robotic navigation via Kubernetes, Istio, and 5G/mmWave networks. The system is validated through a use case in which a ground robot offloads intensive vision tasks to distributed data centers in real time. Their work broadens the design space to include network heterogeneity alongside computing resources.

4.2. Context Awareness

Klaas et al. [50] propose context-aware semantic path optimization for mobile robots that takes advantage of edge computing capabilities inside a distributed microservice-based autonomous control architecture. The objective is to add a semantic layer with an ROS-based modular navigation stack to include significant environmental data from RGB cameras into local path planning. Convolutional Neural Networks (CNNs) are used to semantically classify RGB pictures. In order to improve the quality of the computed path from ordinary local planners, their approach integrates policies based on the semantic classification of the robot's surroundings. This work only emphasizes its importance by exploiting edge computing for the goal of embedding semantics into navigation stacks, but no significant improvement in decreasing operational time is assessed.

Likewise, the same authors, Lambrecht et al. [51], present a micro-service-based architecture for autonomous mobile robots that offloads the entire navigation stack towards the edge. The main advantage of edge computing in terms of energy consumption and on-board computational requirements; however, offloading of navigation takes longer operation time.

The authors Antevski et al. [52] present an advanced control algorithm in MEC to control the robot speed remotely. They begin by running a series of tests to determine the relationship between robot control latency and Wi-Fi signal intensity. The resultant characterization is utilized as a starting point for creating, implementing, and testing a control algorithm that uses context information from the Wi-Fi signal to adjust the robot's speed for smoother driving. The results of their experiments indicate that adjusting the robot's speed depending on the Wi-Fi signal given by the MEC information service may successfully generate smoother driving at high speeds. This enhancement enables the robot to perform faster than if no context information is consumed. The proposed approach advances the real-time adaptation of edge-robot control systems from compute centric approach to a communication context-aware autonomy.

A context-aware testbed architecture, COTORRA, proposed by Groshev et al. [53] operates on the edge of the network. In addition to offering deployment of serverless plugins, it supports testing of time-sensitive applications. It can also emulate unpredictable network conditions that enable a realistic environment where plugins can be tested. They validate through experiments the feasibility of their architecture by running an orchestration algorithm that interacts with the network infrastructure and mobile robots and perform autonomous navigation that improves task latency to less than 15 ms. The authors claim

COTORRA to be the first edge–fog-based robotic testbed of its kind. The approach solidifies testing of context-aware edge-based robotic control under variable network conditions compared to specific controlled conditions employed by previous works.

In the 6G communication domain, Zeng et al. [54] present a semantic communication (SemCom) framework tailored to robotic edge–AI systems where a robot collaborates with a remote edge server acting as its “brain”. The system uses a Knowledge Graph (KG) to represent task-relevant object-action sequences, known as Knowledge Paths (KPs), enabling the server to guide the robot in achieving specific goals based on its environmental observations. A key innovation is a Robotic SemCom protocol that includes semantic matching between KG elements and server-side classifiers, enabling the robot to stream compressed feature data to the server for real-time object recognition and path identification. This work primarily meets ultra-low-latency requirements through a feature transmission scheme (ULL-FT) that leverages the classifier’s robustness to tolerate channel errors, achieving a communication-efficient balance between reliability and speed. The effectiveness of the proposed approach is demonstrated using synthetic and real-world datasets, showing significant latency reductions without compromising inference accuracy. Overall, the work pioneers a goal-oriented air interface for 6G robotic applications where task-driven communication is prioritized over raw data fidelity.

4.3. Localization

According to the authors of [55], it is critical to use edge computing and cloud services to create a well-developed Autonomous Mobile Robot (AMR) system since edge and cloud may minimize AMR power consumption and make integration with sensors or IoT devices easier. They create an efficient AMR system from scratch using an NVIDIA Jetson TX2 as an edge computing power-efficient module that can handle the operation of AMR in unstable or unreliable network conditions, a serverless architecture based on Amazon Web Services (AWS) for cloud computing, and a user interface built in, React Native. They evaluate the performance of their systems by running an indoor delivery service, a facerecognition function for identifying sender and recipient, and caching all requests and responses via cloud-based microservices for rapid task execution.

Many seminal works in tackling the resource constraints posed by local onboard computing in executing SLAM algorithms have turned to edge computing to improve efficiency and reduce task latency. P. Huang et al. present a collaborative multirobot laser SLAM [56] that leverages the edge computing concept for SLAM execution optimization. The robot–edge synergy uses edge computing to enable robots to execute SLAM and produce a global map. Previous measurements on total SLAM offloading on an Alibaba cloud server showed a significant transmission delay. Because feature extraction, matching, and transform optimization are critical submodules of map fusion, the core of multirobot SLAM, they consume more than 70% of the processing time. ColaSLAM addresses this issue by deploying map fusion and mapping modules to edge servers to build a number of submaps and conduct feature extraction and matching by recognizing overlaps between these maps, resulting in map fusion. They propose an adaptive map redistribution approach to speed up map fusion among several edge servers.

A coordinator generates an undirected graph composed of submaps while taking into account the overlapping degree and bandwidth condition, then splits the graph and generates an offloading decision in which each edge cluster is assigned a submap for execution, and finally the edge server merges the received sub-maps and uploads the results to the cloud center to minimize the latency from data collection to the cloud side. Experiments on simulation and prototype evaluate their results on workload balancing against greedy and random approaches and latency, and realize that the proposed algorithm

can reduce the processing latency up to 40% compared to the direct offloading to the cloud and brings up to 52%, on average, improvement compared to random and up to 15% against the greedy algorithm.

Same authors propose RecSLAM [57], a hierarchical robot–edge–cloud system designed to enable low-latency, multi-robot laser SLAM. It addresses the limitations of traditional SLAM approaches—such as excessive local computation and high cloud offloading latency—by distributing SLAM tasks across nearby edge servers. Through real-world experiments and simulations, the system demonstrates significant improvements in efficiency and latency reduction.

Liu et al. [58] propose a centralized multi-robot SLAM system using a robot–edge–cloud architecture, where edge servers handle lightweight optical flow tracking between non-keyframes by receiving only keypoints, reducing computation on robots. Keyframes are compressed and sent to the cloud for pose estimation and map fusion. This approach minimizes communication load while maintaining SLAM accuracy.

Researchers Sarkar et al. in [59] solve the SLAM optimization issue with energy efficiency and performance guarantees as key objectives by leveraging both the edge and fog layers for computationally heavy tasks while leaving the cloud primarily for monitoring, control, and visualization. They suggest a system architecture with four layers: a robot layer and an edge layer for data processing, a fog layer for distributed storage, and a cloud layer for monitoring and general mission control. When the robot moves from one edge to another, the edge layer takes a real-time decision to reduce delay by sharing the computing load of the robot, while the fog layer conducts a handover mechanism with negligible data loss.

This approach includes exchanging prior map data created by the robot during SLAM edge handover. Robots save energy by sending data to the gateway, i.e., the edge and fog layer, rather than directly to the cloud. They observe decreased execution delay due to a shorter data path across the entire cycle, improvement in network bandwidth as possible bottlenecks are addressed by avoiding robot-to-cloud data transmission, and considerable energy improvement on the robots through trials. Although they conduct real-world testing on a small automobile prototype, the results are insufficient to demonstrate the efficacy of the suggested method. Perhaps some stress testing on a real-world multi-robot system may aid in further evaluating the algorithm.

A 3D semantic map creation approach for mobile robots based on enhanced ORB-SLAM2 is proposed by Cui et al. [60] to ensure that mobile robots have a certain capacity to perceive targets in indoor environments and to improve the degree of intelligence of robots. The proposed technique involves first the implementation of an improved version of YOLOv3 to detect indoor objects, then a real-time semantic segmentation network model is employed based on deep learning that segments the objects to classify their pixel points on a two-dimensional image, and to improve the accuracy of semantic segmentation, the BAFF feature fusion algorithm is applied. A 3D semantic map is then created by the use of SLAM that estimates the pose of the image as a result of semantic segmentation. Using this depth information, it is projected onto the 3D environment.

Solving the resource optimization problem on robots, the authors Ben Ali et al. [61] propose the implementation of ORB-SLAM2 using edge computing to offload parts of Visual-SLAM. They split the architecture of the algorithm between the edge and the robot by keeping the tracking module on the robot while moving more intensive computational modules, i.e., local mapping and loop closure, to the edge. By keeping the components tightly connected, they hope to achieve their goal of lowering computational and memory overhead without compromising performance precision.

Because the local mapping and loop closing modules rely on the global map for some of their computation, it is critical to update the local map whenever the global map changes, necessitating close communication between the modules, which is accomplished by introducing three separate network connections. The first two connections are responsible for obtaining data from the tracking module; i.e., one where the tracking module communicates the features and local geometry in *Frames* and the other passes the keyframe if tracking decides to create a new keyframe, which is used for creating and storing the global map on the edge. The third connection is for updating the local map on the mobile robot and a map synchronization module on the edge that synchronizes the local map updates with the latest optimized changes.

Upon implementation on a prototype through rigorous experiments, the split architecture proposed reduces computation and memory cost while keeping the execution time to a minimum. The latency involved with an update of the local map between the robot and the edge also substantially reduces the overall VSLAM execution time. Performance is estimated through mapping accuracy by mean localization error, observed negligible loss of accuracy in the final map, and the trajectory taken.

Another work by Chen et al. [62] investigates edge-computed aided autonomous flight for UAV (ECAAF), in which vision and communication modules deployed in the framework interact and support each other using edge computing and offloading to speed up the UAV mission. ECAAF has three functionalities: 3D map acquisition with an edge layer, radio map generation from the 3D map, and live trajectory planning. A positive feedback loop also allows interaction between edges and UAVs for video offloading, 3D map quality, and channel state of the trajectory form. The authors test their suggested strategy using simulations which show that it can increase mission performance by boosting interconnectivity.

4.4. Navigation

Many works in the literature attempt to address the MRS navigation issue using edge computing. Palani et al. [63] propose an edge computing-based autonomous robot, where the edge is deployed for data processing to calculate the path to the source of product boxes in a manufacturing industry scenario to avoid accidents involving product damage. They develop a six-degree-of-freedom manipulator in SolidWorks and use it to test a navigation algorithm in an obstacle-filled environment.

The edge computing module used in their suggested system intends to solve the process of discovering the path for navigation utilizing a small quantity of data at a fast speed and with minimal time delay by employing obstacle detection and avoidance, as well as enabling direct control. Experiments are carried out to verify the accuracy of direction tracking and the time of operation of the robot, but no important conclusions can be derived because the research lacks comparison with benchmark methodologies, limiting the study's impact.

Fan et al. [64] offer a collaboration method based on an edge computing framework, a robot "StellaX," and a manipulator in an attempt to accomplish navigation and object identification for grasping through cooperation across heterogeneous robots. The mobile robot "StellaX" features three omnidirectional wheels and is outfitted with a LIDAR scanner for SLAM and navigation, as well as a stereo camera for object identification and grasping. The manipulator and the robot communicate via edge servers, where information about the object used for grasping is saved and transferred so that the manipulator may access it. Docker Edge Robotics Framework (DERF) is a container engine technology based on Linux container (LXC) that provides the benefits of quick and efficient deployment, high resource

utilization, and simple administration. DERF is divided into the following categories: service provider, service requester, cloud center, edge node, and physical layer.

A Docker container image is built and registered with the cloud center in order to carry out the navigation (*gmapping* with *amcl* ROS package as described in the article). As the requester, the user can send the service request over the network, and once received, the management module in the cloud center performs a service query in the image repository to confirm that the service exists and the robot is idle, after which the image is instantiated to form a functional application that sends task instructions to the physical layer via the edge node. Experiments on object identification and navigation demonstrate the differences between local, cloud, and edge computing. The entire execution time is greatly decreased by using edge computing, and the upload time for edge computing is always less than that of the cloud. The use of the edge layer also considerably decreases CPU and memory usage rates, as well as navigation execution time.

Authors Li et al. [65] propose a visual navigation algorithm implemented on the edge computing platform for agricultural robots based on deep learning image understanding. To process pictures acquired by the proposed vision system, the approach initially employs a cascaded deep convolutional network and a hybrid dilated convolution fusion algorithm. The modified Hough transform technique is then used to extract the path from processed pictures. At the same time, the agricultural robot's posture is being adjusted to enable autonomous navigation. Their proposed method is validated using non-interference and noisy experimental scenes in real-world experiments. The results of the experiments reveal that the algorithms proposed perform better in the non-interference scene and complex noise scenes, but no experimental data can be obtained from using an edge computing platform or any knowledge of whether their objectives are satisfied through the use of edge devices.

Qingqin et al. [66] investigate the state of the art in visual odometry-based autonomous navigation for MRS that are confined by computing and sensor resources. Image compression reduces the quality of navigation, but it is not the only method for improving execution time and accuracy. Through this experimental study, the authors discover that image size and network bandwidth may be reduced by an order of magnitude without compromising the accuracy of the odometry methods, even in challenging environments. Their tests look at two parameters: network latency and odometry algorithm accuracy. Using the EuRoC dataset, they examine the processing time required for the feature extraction and posture estimation processes for each of the image compression ratios.

The findings show that reducing picture quality to a certain degree has no effect on odometry accuracy, indicating that the system can benefit from a computational offloading approach using edge computing. They demonstrate through findings that offloading does not cause odometry delays and actually enhances frame rate performance when powerful edge gateways are deployed. They suggest an edge computing offloading strategy that can provide several benefits to a large MRS system, including reduced cost and energy consumption, as well as improved performance and reliability.

Hayat et al. [67] analyze the importance of edge computing in 5G for vision-based navigation algorithms for drones. They deploy edge devices to offload computing tasks for image processing. They test various modes of computational offloading, including onboard, fully offloaded to the edge, and partially offloaded. Their findings reveal that for all image resolutions, the image processing time for complete offloading is longer than for partial offloading, and this difference becomes more pronounced as resolution increases. When the computational power of the server is twice that of the drone, partial offloading results in faster image processing times than onboard processing. This is not true for

complete offloading, however. Full offloading necessitates long image transmission times, and increased edge compute power has little effect on overall image processing time.

The authors then investigate the effect of 5G data rates on image processing time in each mode. They conclude that higher up-link speeds can meet the image transmission requirements of the full offloading mode. Extracting features onboard and sending them to the edge provides more advantage than delivering the entire image. They also highlight that partial offloading places less strain on the communication network in terms of transmission rate than full offloading, but still needs some onboard processing.

Li et al. [68] propose that the Edge Accelerated Robot Navigation (EARN) framework enhances robot navigation by adaptively switching between local and edge-based motion planning based on resource availability. Unlike traditional approaches that focus solely on algorithm design, EARN integrates decision-making and motion planning under communication and computation constraints using model predictive switching. It effectively utilizes edge resources to enable advanced collision avoidance and optimized path planning. While the framework demonstrates strong performance in both simulated and real-world settings, its reliance on low-latency edge connectivity and complex optimization algorithms may pose deployment challenges in dynamic or resource-constrained environments.

4.5. Minimizing Task Latency

Minimizing task latency is largely attributed to improving the network conditions in the edge computing domain. Asavasirikulkij et al. [69] devise a new peer-to-peer wireless communication platform for communication between a mobile robot and an edge device in a smart factory scenario. The application tested includes a mobile robot equipped with sensors and connected to an edge device and a serial link manipulator. Through their proposed wireless platform that offers high bandwidth and low communication latency via the use of a Software-Defined Wireless Networking (SDWN) approach, they implement a pose estimation through SLAM on the edge device based on LIDAR scan data transmitted from the robot to the edge.

They extend their work further by proposing a wireless network connecting several machines in an automation cell, i.e., a robot, a manipulator, a warehouse, and an edge server. The edge is deployed to collect all the data from the machines in-house to generate an updated map for controlling the automation cell during robot collaboration. They test their proposed setup through a use case scenario where the mobile robot picks up a package from the manipulator and delivers it to the warehouse using the pose estimates provided by the edge device in the automation cell. Experimental results evaluate the collaborative map generated as well as the path execution comparison between the planned path and the mobile robot's actual path used for navigation. Compared to two other benchmark techniques, their proposed architecture undergoes less processing latency to compute SLAM.

Another work put forth by Yin et al. [70] elaborates on a MEC-based multirobot cooperation (MRC) system with Primary Robot (PR) and Secondary Robots (SRs). They envision the PR as an edge device in charge of making decisions based on sensing conducted by the SRs. In addition to sensing data, the SRs offload various functions to the PR in order to reduce energy utilization onboard. The research provides two resource management solutions for minimizing and balancing energy usage across multiple SRs while ensuring task completion on time. The energy consumption of the SRs is minimized and balanced in the first scheme to increase the function duration of the MRC systems. The residual energy of the SRs is employed in the second scheme to deal with the unpredictable wireless communication environment. As a result, instead of transferring the data to the PR, each SR may handle it locally. The latter scheme is more durable than the former, although it

may consume more energy. They test their methods using simulations, which show that MEC can help MRC handle time-critical heavy computing tasks effectively.

The mobility of robots poses an additional challenge for computational offloading, particularly when multi-robot systems depend on real-time decision-making from AI systems deployed on edge servers. Frequent access point handovers lead to service migration between edge servers, resulting in increased latency and negatively impacting the performance of tasks such as autonomous navigation.

Tahir and Parasuraman [71] propose a deep reinforcement learning framework based on the Deep Deterministic Policy Gradient (DDPG) algorithm to address the joint problem of task migration and access-point handover in vehicular networks. They introduce a joint allocation method for communication and computation across access points to minimize computational load, service latency, and interruptions, with the goal of maximizing Quality of Service (QoS). Their framework, evaluated through simulated experiments, enables smooth task switching among edge servers, reducing latency and improving service efficiency.

The authors K. Chen et al. [72] introduce the FogROS2-Latency-Sensitive (FogROS2-LS) framework, which dynamically selects the most suitable cloud or edge server to minimize latency for robots offloading computationally intensive tasks. Their framework offloads conventional on-board state estimators and feedback controllers to cloud and edge devices without requiring modifications to existing ROS2 applications. When multiple identical services are available, FogROS2-LS dynamically identifies and switches to the optimal service deployment that meets latency requirements, enabling robots with limited on-board computing power to safely and efficiently navigate dynamic, human-populated environments.

Because the majority of the works in the literature involving computational offloading to edge computing focus on latency minimization, either in terms of execution time or transmission time, such studies are not repeated in this section for the sake of brevity.

4.6. Resource Optimization

Qingqing et al. [73] suggest using FPGAs at the edge for computational offloading with low latency and high parallelism in a multirobot feature-based LIDAR odometry system. In three phases, their approach performs feature-based LIDAR odometry: The extraction of features from LIDAR data comes first. These characteristics might be geometric in form or stable points observed in two successive sweeps. Second, it employs feature correspondence by calculating the position difference between the sweeps.

Finally, the time interval between two successive sweeps is used to estimate the LIDAR movement. The purpose of employing FPGAs is to process scan data in real time with minimum computational resources and to parallelize data processing. The authors compare their technique to their own initial and unoptimized design, in terms of resource usage and concurrency gained, which was shown to be superior for the optimized proposed version.

Spatharakis et al. [9] provide a set-based estimation approach for the robot offloading mechanism in the context of edge robotics. Using a case study of a unicycle robot performing navigation and path planning, they assess the tradeoff between performance and resource consumption. An offloading strategy is introduced to compensate for the uncertainty of local estimation techniques with more accurate remote sensing, while balancing navigation accuracy and mission duration. While traversing its environment, the unicycle robot uses remote estimates to correct inaccuracies in local estimates. Offloading is performed via a framework that considers network conditions and computational availability while maintaining the stability of the control system.

Their decision-making computational offloading strategy is framed as a utility-based function that incorporates two localization techniques: an error-prone one, i.e., odometry-based localization (executed locally), and a more accurate, computationally intensive vision-based localization method executed on the edge server. The authors compared their utility-based offloading scheme with two others—exclusively local implementation and exclusively remote—in terms of average mission duration (in seconds), average offloading triggers, and success rate. They also presented a comparison of their utility-based offloading scheme under five different time-triggered settings, where remote estimation is triggered every μ seconds across a total of 35 experiments. Their technique outperformed all others by providing convergence guarantees while keeping mission duration low.

Huang et al. [74] develop an offloading service provider in Collaborative Vehicular Edge Computing (CVEC) system by launching an MEC server and scheduling PVs (parking vehicles) on demand to perform offloading tasks. To optimize network-wide task scheduling, efficient workload allocation and user-centric utility maximization are studied. Each task's offloading destination is determined probabilistically in a dynamic context. When necessary, the offloading service provider acts in the capacity of an offloading user to initiate a contract-based incentive mechanism for PVs. The authors base the offloading user's subjective assessments of utility in computational offloading using contract theory and prospect theory and develop an ideal contract to maximize subjective utility under formation asymmetry. Numerical data collected through experiments indicate the proposed scheme's efficacy and efficiency.

Bouhoula et al. [75] present a DRL-based task scheduling strategy designed for MEC-enabled robotic systems in dynamic environments by jointly optimizing task execution, robot trajectory planning, and energy consumption, addressing the challenges caused by robot mobility, transmission delays, and resource constraints. The authors transform the problem into a Markov Decision Process (MDP), enabling real-time adaptive decisions such as local processing versus offloading, and device-to-device (D2D) task sharing. Their extensive evaluations demonstrate significant improvements over baseline methods in reducing delays and energy use. While the approach excels in scalability and adaptability, it remains dependent on simulation scenarios and may face challenges in deployment complexity in the real world.

4.7. Minimizing Energy Consumption

Wang et al. [76] propose a remote computational offloading mechanism for a swarm of robots connected to an edge server. The primary goal of their method is to reduce energy consumption and task execution time. They develop a deep reinforcement learning system for task scheduling that takes the robot's mobility into account. The authors show the efficiency of their technique in decreasing energy consumption and delay through simulation. However, their system paradigm is based on a single edge server for management and compute offloading, rendering the study incomplete with respect to the deployment of multiple edge devices and their impact on the performance.

Farahbaksh et al. [77] present a computational offloading approach based on Bayesian learning automata for Mobile Edge Computing. Their method incorporates contextual information awareness which influences decision-making for computational offloading from mobile devices to edge servers. Context information is captured via a control loop. Several metrics are studied to evaluate the proposed scheme, including energy consumption, execution time, resource utilization, network usage, task latency, module size, and offloading intervals.

Authors Zeng et al. [78] address the problem of aggregated and centralized training which strains on long-range backhaul transmission. They propose a collaborative edge

training framework that leverages idle resources from trusted edge devices as a resource pool for efficient and sustainable training of large AI models. The authors design the system architecture, analyze scheduling strategies for energy optimization, and evaluate different parallelism methods using realistic testbeds.

There is some overlap among the aforementioned efforts that address energy consumption using edge robots. Although not exhaustive, this list omits certain works for brevity.

5. Research Gaps and Limitations

Real-world evaluations of edge-assisted robot systems are critical for validating performance, yet many existing studies exhibit significant limitations in scope, generalizability, and experimental rigor. For example, S. Dey et al. [43] conduct comprehensive hardware evaluations but relied solely on virtual machines as edge devices, with no physical robot deployment. Their experiments assumed idealized network conditions—constant bandwidth, uniform processing capabilities, and negligible variability—thereby overlooking crucial factors such as real-time network dynamics, energy constraints, and fault tolerance that are essential for practical SLAM offloading in mobile robotics.

Similarly, the NETMODE testbed by D. Dechouniotis et al. [44] employs Raspberry Pi 3 units as mobile robots, which offer limited computational capacity. While the system evaluates CPU usage, it omits other essential performance indicators such as latency, task completion success, localization accuracy, and energy consumption. Moreover, their manually configured architecture lacks support for dynamic task orchestration, scalability to multi-robot systems, or resource-aware decision-making at the edge—severely limiting its applicability to complex, long-term, or large-scale robotic deployments.

In another instance, the cloud-edge hybrid architecture proposed in [45] reduces latency by offloading lightweight AI inference to the edge while relegating heavy model training to the cloud. While effective for improving processing speed, the system's dependence on stable connectivity and the limited computational capacity of the edge restrict its robustness and scalability in dynamic environments, especially those involving multiple robots or unpredictable wireless conditions.

H. Wang et al. [46] evaluate performance under static terminal assumptions, predictable task distributions, and idealized communications, which fail to reflect the nuanced variability of bandwidth, mobility, and hardware resource contention inherent in real-world mobile robotic systems.

Baruffa et al. [49] present a modular testbed integrating 5G, mmWave, and cloud-edge infrastructure for robotic navigation, yet the system relies on specialized infrastructure and evaluates only a narrow slice of performance metrics. Notably, it lacks comprehensive testing for scalability, security, and adaptability to diverse deployment scenarios. Likewise, the work by J. Lambrecht et al. [51] shows impressive CPU and RAM offloading benefits and up to 38.59% energy savings during navigation and vision tasks. However, these benefits are measured under controlled conditions—fixed robot velocity, no payload, and static workload—without accounting for edge-side energy costs or network-induced overheads. Moreover, the system lacks stress testing under fluctuating workloads, making it difficult to generalize to unpredictable operational contexts.

The COTORRA testbed [53] suffers from reliance on synthetic network variability via NetEm, using artificially injected delays rather than real-world wireless channel dynamics. Conducted in constrained indoor corridors, the experiments omit outdoor, user-interactive, or heterogeneous multi-robot environments. Orchestration evaluations are limited to a strict 15 ms latency target without assessing robustness against threshold violations, and the DLT federation component exhibits a prohibitively high 19-second delay—unsuitable for

time-sensitive applications. Robot autonomy is limited to pre-programmed paths, lacking responsiveness to environmental complexity or uncertainty.

The ColaSLAM system [56] successfully demonstrates collaborative multi-robot mapping using tiered edge-cloud fusion, significantly reducing on-robot computational load. However, it assumes uninterrupted wireless connectivity and performs no local data preprocessing, increasing transmission demands and vulnerability to network disruptions. Additionally, the system's coordinator operates intermittently, which may introduce scheduling delays in dynamic conditions. Real-world tests also reveal mapping inaccuracies at boundaries and dependence on precise overlap detection, both of which can hinder map fusion reliability.

Finally, the Edge-SLAM framework by A. J. Ben et al. [61] is evaluated using a minimal hardware setup involving only two mobile devices and one edge server, which limits the generalizability of results to more complex settings. The use of replayed datasets over private Wi-Fi networks and a wired edge connection fails to capture performance under variable mobility, fluctuating bandwidth, or real-time wireless interference. While Edge-SLAM demonstrates accuracy comparable to ORB-SLAM2, architectural decomposition and latency could exacerbate drift in longer or more complex deployments. Moreover, tracking sensitivity issues inherited from ORB-SLAM2 remain unaddressed, highlighting gaps in robustness for real-world SLAM scenarios.

As evident from the critical examination of existing experimental frameworks, edge robotics has made substantial strides through innovative system architectures, distributed offloading mechanisms, and the integration of heterogeneous computing platforms. These developments have improved real-time processing for tasks such as SLAM and autonomous navigation, particularly in controlled environments. However, the evaluation of these systems often falls short of capturing the operational complexity of real-world deployments. Many experimental setups rely on idealized assumptions—such as stable, high-throughput wireless networks, deterministic robot behaviors, or fixed environmental layouts—that do not reflect the variability and uncertainty encountered in practical field conditions. Furthermore, reliance on small-scale testbeds, synthetic data replay, and limited performance metrics hinders the ability to assess scalability, fault tolerance, and adaptability under stress. Deep learning and AI-based methods, though promising, add layers of computational and interpretive complexity that are rarely stress-tested under constrained edge resources. The absence of standardized benchmarks, dynamic workload orchestration, and thorough evaluations across diverse scenarios remains a critical bottleneck. To unlock the full potential of edge-enabled robotics, future research must prioritize robust, scalable, and generalizable experimental frameworks—grounded in real-world constraints—that can systematically address the challenges of interoperability, resilience, and continuous adaptation across multi-robot and multi-environment systems.

6. Challenges for Future Research

Computational offloading for robots offers a multitude of benefits for ground and aerial robots in real-world deployments. It allows for leveraging the computational and storage facilities available at the remote resources to perform latency-sensitive and computationally intensive operations, particularly algorithms involving deep learning or computer vision, and it also allows for maintaining communication between its distributed platform.

However, network conditions associated with such frameworks introduce implementation challenges related to delay and energy consumption. Offloading is a critical step in increasing the quality of experience for robots and facilitating the execution of applications with low latency requirements. Undoubtedly, significant efforts were made to increase the robot's performance via computational offloading on edge-based systems. Several

challenges, however, remain open study areas for further investigation. Some of these problems are discussed more below.

6.1. Security

Security [79] is a key performance parameter in all networked robotic applications. Communication and data transmission between robots and distant computing resources can lead to security concerns such as modification of data, denial of service, and information leakage [80]. Unfortunately, in the studies mentioned above, security was not considered a primary optimization objective. Like other objectives stated, the security aspect should be incorporated as an key decision-making parameter to mitigate risks associated with data vulnerabilities.

6.2. Context Awareness

As evident from Table 2, context awareness has received limited attention in the design of offloading mechanisms, despite improvements in related works. Given that we are discussing dynamic computational offloading, context awareness is a critical aspect that warrants deeper exploration. It should inform every stage of the offloading process. For example, the program partitioning stage must be aware of the application's resource requirements to offload. The offloading decision should therefore be informed by the context information, such as distant server availability, network quality, and device workload, at all times [81].

Table 2. Comparison of edge robotics applications based on objectives.

Reference	Computational Offloading	Context Awareness	Localization	Navigation	Minimizing Latency	Resource Optimization	Minimizing Energy Consumption
S. Dey et al. [43]	✓	✗	✓	✗	✓	✗	✓
D. Dechouniotis et al. [44]	✓	✗	✓	✓	✓	✓	✗
W. Li et al. [45]	✓	✗	✗	✗	✗	✗	✓
H. Wang [46]	✓	✗	✗	✗	✓	✓	✓
N. Tahir et al. [47,48]	✓	✗	✓	✓	✓	✓	✗
Baruffa et al. [49]	✓	✗	✗	✓	✓	✓	✗
T. Klaas et al. [50]	✓	✓	✗	✓	✗	✗	✗
J. Lambrecht et al. [51]	✓	✓	✗	✓	✗	✗	✓
K. Antevski et al. [52]	✗	✓	✗	✓	✓	✗	✗
M. Groshev et al. [53]	✓	✓	✗	✗	✓	✗	✗
Q. Zeng et al. [54]	✓	✓	✗	✗	✓	✓	✗
T. Thong Tran et al. [55]	✓	✗	✓	✓	✓	✗	✓
P. Huang et al. [56,57]	✓	✗	✓	✗	✓	✓	✗
Lui et al. [58]	✓	✗	✓	✗	✓	✓	✗
V. Sarkar et al. [59]	✓	✗	✓	✗	✓	✓	✓
X. Cui et al. [60]	✗	✗	✓	✗	✗	✗	✗
A. Ben Ali et al. [61]	✗	✗	✓	✓	✓	✗	✗
Q. Chen et al. [62]	✗	✗	✓	✓	✓	✗	✗
U. Palani et al. [63]	✗	✗	✗	✓	✓	✗	✗
Z. Fan et al. [64]	✗	✗	✓	✓	✓	✗	✗
J. Li et al. [65]	✗	✗	✗	✓	✗	✗	✗
L. Qingqing et al. [66]	✓	✗	✓	✓	✓	✗	✗
S. Hayat et al. [67]	✓	✗	✓	✓	✓	✗	✓

Table 2. Cont.

Reference	Computational Offloading	Context Awareness	Localization	Navigation	Minimizing Latency	Resource Optimization	Minimizing Energy Consumption
G. Li et al. [68]	✓	✗	✗	✓	✓	✓	✗
C. Asavasirikulkij et al. [69]	✗	✗	✓	✓	✓	✗	✓
R. Yin et al. [70]	✓	✗	✗	✗	✓	✓	✓
N. Tahir et al. [71]	✓	✗	✗	✓	✓	✓	✗
K. Chen et al. [72]	✓	✗	✗	✗	✓	✓	✗
L. Qingqing et al. [73]	✓	✗	✗	✗	✓	✓	✗
D. Spatharakis et al. [9]	✓	✗	✓	✓	✓	✓	✗
X. Huang et al. [74]	✓	✓	✗	✗	✓	✗	✗
S. Bouhoula et al. [75]	✓	✗	✗	✗	✓	✓	✓
Wang et al. [76]	✓	✗	✗	✗	✓	✗	✓
F. Farahbaksh et al. [77]	✓	✓	✗	✗	✓	✗	✓
Zeng et al. [78]	✓	✗	✗	✗	✗	✓	✓

Legend: ✓ Yes indicates that the objective is fully or partially explored in the research; ✗ No indicates that the objective is not considered.

6.3. Handover Mechanism

During the crossover between multiple edge devices, there is a huge possibility of data loss. While the task finishes itself on one edge device and switches to another, the robots either enter into an idle state or halt their execution, which can introduce latency and can also become costly and pose a safety hazard while performing tasks like navigation. All dynamic computational offloading schemes must be designed to be equipped with an efficient handover mechanism to avoid computational cost, latency, or task failures.

6.4. Network Failure

Connectivity loss is another major concern that can happen during offloading, where the robots end up losing connection with the remote resources and keep waiting for the communication to resume when it could be a complete network failure or a temporary connectivity loss. Accordingly, it is beneficial if the robots take charge of the execution by performing the tasks locally, which should work ideally but can be a serious problem in performing algorithms that have been split among resources to save on energy consumption and execution latency.

6.5. Fault Tolerance

The dependence on network circumstances is one of the most crucial issues in the computation of offloading for mobile robots. This might result in a failure of the offloading process, either due to a weak connection between robots and remote resources or an issue with the remote edge server. Because mobile robots are intended to be used in critical scenarios like search and rescue missions, it is critical to avoid these problems and ensure the application's proper execution. Other alternatives must be devised whereby the robots can switch to local execution or offload their tasks to other emergency backup remote resources to avoid any delay in task execution.

6.6. Heterogeneity

Although heterogeneity is a desirable feature in the implementation of the edge computing domain, its integration with robots can cause major interoperability concerns, which can be a hurdle in the effective deployment of an edge computing framework in a heterogeneous or otherwise multirobot system.

All such failures that might occur during task offloading of computational-intensive tasks lead to either increased energy consumption or increasing latency time for task execution or data transmission. An efficient offloading strategy must take into account all such factors to make informed decisions or provide remedial solutions as part of the recovery mechanism; therefore, it is suggested that fault tolerance and recovery strategies must be an essential part of the offloading decision-making process.

7. Conclusions

This study presents a comprehensive review of the current state of the art in the edge robotics domain. While it offers a broad overview, it is not exhaustive and can be extended further, particularly by incorporating a detailed classification of the literature based on fog computing technologies. The findings highlight that although many studies have leveraged edge platforms to address the computational demands of robotic applications, the field remains in its early stages. There is significant potential to pursue overlapping objectives and address persistent challenges related to the practical implementation of edge computing in robotics. These limitations—such as efficient resource management, scalable system architectures, robust failure recovery mechanisms, and strong security provisions—can be overcome through well-designed solutions tailored to the unique requirements of edge robotic systems.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

Acronym	Description
ETSI	European Telecommunications Standards Institute
MEC	Mobile Edge Computing
CPS	Cyber-Physical Systems
SLAM	Simultaneous Localization and Mapping
AWS	Amazon Web Services
ROS	Robot Operating System
MRS	Multi-Robot Systems
RAN	Radio Access Network
FCN	Fog Computing Nodes
OS	Operating System
VM	Virtual Machines
GPU	Graphics Processing Unit
NPU	Neural Network Processor Unit
FGPA	Field Programmable Gate Array
TPU	Tensor Processing Unit
APU	Accelerated Processing Unit
UAV	Unmanned Aerial Vehicle
LSTM	Long Short-Term Memory
MRS	Multi-Robot System

References

1. Sun, X.; Ansari, N. EdgeIoT: Mobile edge computing for the Internet of Things. *IEEE Commun. Mag.* **2016**, *54*, 22–29. [[CrossRef](#)]
2. Alrawais, A.; Alhothaily, A.; Hu, C.; Cheng, X. Fog computing for the internet of things: Security and privacy issues. *IEEE Internet Comput.* **2017**, *21*, 34–42. [[CrossRef](#)]
3. Hassan, N.; Gillani, S.; Ahmed, E.; Yaqoob, I.; Imran, M. The role of edge computing in internet of things. *IEEE Commun. Mag.* **2018**, *56*, 110–115. [[CrossRef](#)]

4. Khan, W.Z.; Ahmed, E.; Hakak, S.; Yaqoob, I.; Ahmed, A. Edge computing: A survey. *Future Gener. Comput. Syst.* **2019**, *97*, 219–235. [\[CrossRef\]](#)
5. Zhao, Z.; Liu, F.; Cai, Z.; Xiao, N. Edge computing: Platforms, applications and challenges. *J. Comput. Res. Dev.* **2018**, *55*, 327–337.
6. Maskeliūnas, R.; Damaševičius, R.; Segal, S. A review of internet of things technologies for ambient assisted living environments. *Future Internet* **2019**, *11*, 259. [\[CrossRef\]](#)
7. Cao, K.; Liu, Y.; Meng, G.; Sun, Q. An overview on edge computing research. *IEEE Access* **2020**, *8*, 85714–85728. [\[CrossRef\]](#)
8. Chen, G.; Kang, B.T.; Kandemir, M.; Vijaykrishnan, N.; Irwin, M.J.; Chandramouli, R. Studying energy trade offs in offloading computation/compilation in java-enabled mobile devices. *IEEE Trans. Parallel Distrib. Syst.* **2004**, *15*, 795–809. [\[CrossRef\]](#)
9. Spatharakis, D.; Avgeris, M.; Athanasopoulos, N.; Dechouniotis, D.; Papavassiliou, S. Resource-aware estimation and control for edge robotics: A set-based approach. *IEEE Internet Things J.* **2022**, *10*, 2003–2020. [\[CrossRef\]](#)
10. Tahir, N.; Parasuraman, R. Analog twin framework for human and ai supervisory control and teleoperation of robots. *IEEE Trans. Syst. Man Cybern. Syst.* **2022**, *53*, 2616–2628. [\[CrossRef\]](#)
11. Hu, G.; Tay, W.P.; Wen, Y. Cloud robotics: Architecture, challenges and applications. *IEEE Netw.* **2012**, *26*, 21–28. [\[CrossRef\]](#)
12. Wan, J.; Tang, S.; Yan, H.; Li, D.; Wang, S.; Vasilakos, A.V. Cloud robotics: Current status and open issues. *IEEE Access* **2016**, *4*, 2797–2807. [\[CrossRef\]](#)
13. Rahman, A.; Jin, J.; Rahman, A.; Cricenti, A.; Afrin, M.; Dong, Y.n. Energy-efficient optimal task offloading in cloud networked multi-robot systems. *Comput. Netw.* **2019**, *160*, 11–32. [\[CrossRef\]](#)
14. Afrin, M.; Jin, J.; Rahman, A.; Rahman, A.; Wan, J.; Hossain, E. Resource allocation and service provisioning in multi-agent cloud robotics: A comprehensive survey. *IEEE Commun. Surv. Tutor.* **2021**, *23*, 842–870. [\[CrossRef\]](#)
15. Groshev, M.; Baldoni, G.; Cominardi, L.; de la Oliva, A.; Gazda, R. Edge robotics: Are we ready? An experimental evaluation of current vision and future directions. *Digit. Commun. Netw.* **2023**, *9*, 166–174. [\[CrossRef\]](#)
16. Haidegger, T.; Galambos, P.; Rudas, I.J. Robotics 4.0—Are we there yet? In Proceedings of the 2019 IEEE 23rd International Conference on Intelligent Engineering Systems (INES), Gödöllő, Hungary, 25–27 April 2019; pp. 000117–000124.
17. Ayranci, A.A.; Erkmen, B. Edge computing and robotic applications in modern agriculture. In Proceedings of the 2024 International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), Istanbul, Türkiye, 23–25 May 2024; pp. 1–6.
18. Sharma, M.; Tomar, A.; Hazra, A. Edge computing for industry 5.0: Fundamental, applications, and research challenges. *IEEE Internet Things J.* **2024**, *11*, 19070–19093. [\[CrossRef\]](#)
19. Wan, S.; Gu, Z.; Ni, Q. Cognitive computing and wireless communications on the edge for healthcare service robots. *Commun.* **2020**, *149*, 99–106. [\[CrossRef\]](#)
20. Kuffner, J. *Cloud-Enabled Robots in: IEEE-Ras International Conference on Humanoid Robots*; IEEE: Piscataway, NJ, USA, 2010.
21. Kehoe, B.; Patil, S.; Abbeel, P.; Goldberg, K. A survey of research on cloud robotics and automation. *IEEE Trans. Autom. Sci. Eng.* **2015**, *12*, 398–409. [\[CrossRef\]](#)
22. Arumugam, R.; Enti, V.R.; Bingbing, L.; Xiaojun, W.; Baskaran, K.; Kong, F.F.; Kumar, A.S.; Meng, K.D.; Kit, G.W. DAVinCi: A cloud computing framework for service robots. In Proceedings of the 2010 IEEE International Conference on Robotics and Automation, Anchorage, AK, USA, 3–7 May 2010; pp. 3084–3089.
23. Montemerlo, M.; Thrun, S. FastSLAM 1.0. *FastSLAM: A Scalable Method for the Simultaneous Localization and Mapping Problem in Robotics*; Springer: Berlin/Heidelberg, Germany, 2007; pp. 27–62.
24. Mohanarajah, G.; Hunziker, D.; D’Andrea, R.; Waibel, M. Rapyuta: A cloud robotics platform. *IEEE Trans. Autom. Sci. Eng.* **2014**, *12*, 481–493. [\[CrossRef\]](#)
25. Riazuelo, L.; Civera, J.; Montiel, J.M. C2tam: A cloud framework for cooperative tracking and mapping. *Robot. Auton. Syst.* **2014**, *62*, 401–413. [\[CrossRef\]](#)
26. Sartoni, M. AWS Services for Cloud Robotics Applications. Ph.D. Thesis, Politecnico di Torino, Turin, Italy, 2022.
27. Saha, O.; Dasgupta, P. A comprehensive survey of recent trends in cloud robotics architectures and applications. *Robotics* **2018**, *7*, 47. [\[CrossRef\]](#)
28. Bonomi, F.; Milito, R.; Zhu, J.; Addepalli, S. Fog computing and its role in the internet of things. In Proceedings of the First Edition of the MCC Workshop on Mobile Cloud Computing, Helsinki, Finland, 17 August 2012; pp. 13–16.
29. Gudi, S.C.; Ojha, S.; Clark, J.; Johnston, B.; Williams, M.-A. Fog robotics: An introduction. In Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems, Vancouver, BC, Canada, 24–28 September 2017.
30. Mohamed, N.; Al-Jaroodi, J.; Jawhar, I. Fog-Enabled Multi-Robot Systems. In Proceedings of the 2018 IEEE 2nd International Conference on Fog and Edge Computing (ICFEC), Washington, DC, USA, 1–3 May 2018; pp. 1–10. [\[CrossRef\]](#)
31. Galambos, P. Cloud, fog, and mist computing: Advanced robot applications. *IEEE Syst. Man Cybern. Mag.* **2020**, *6*, 41–45. [\[CrossRef\]](#)
32. Hu, Y.C.; Patel, M.; Sabella, D.; Sprecher, N.; Young, V. Mobile edge computing—A key technology towards 5G. *ETSI White Pap.* **2015**, *11*, 1–16.

33. Giust, F.; Verin, G.; Antevski, K.; Chou, J.; Fang, Y.; Featherstone, W.; Fontes, F.; Frydman, D.; Li, A.; Manzalini, A.; et al. MEC deployments in 4G and evolution towards 5G. *ETSI White Pap.* **2018**, *24*, 1–24.
34. Wang, F.; Zhang, M.; Wang, X.; Ma, X.; Liu, J. Deep learning for edge computing applications: A state-of-the-art survey. *IEEE Access* **2020**, *8*, 58322–58336. [[CrossRef](#)]
35. Afrin, M.; Jin, J.; Rahman, A. Energy-delay co-optimization of resource allocation for robotic services in cloudlet infrastructure. In Proceedings of the International Conference on Service-Oriented Computing, Hangzhou, China, 12–15 November 2018; Springer: Berlin/Heidelberg, Germany, 2018; pp. 295–303.
36. Taleb, T.; Samdanis, K.; Mada, B.; Flinck, H.; Dutta, S.; Sabella, D. On multi-access edge computing: A survey of the emerging 5g network edge cloud architecture and orchestration. *IEEE Commun. Surv. Tutor.* **2017**, *19*, 1657–1681. [[CrossRef](#)]
37. *IEEE P1934/D1.0*; IEEE Draft Standard for Adoption of Openfog Reference Architecture for Fog Computing. IEEE: Piscataway, NJ, USA, February 2018; pp. 1–170.
38. Pujol, V.C.; Dustdar, S. Fog robotics—Understanding the research challenges. *IEEE Internet Comput.* **2021**, *25*, 10–17. [[CrossRef](#)]
39. Gudi, S.L.K.C.; Johnston, B.; Williams, M.-A. Fog robotics: A summary, challenges and future scope. *arXiv* **2019**, arXiv:1908.04935. [[CrossRef](#)]
40. Song, D.; Tanwani, A.K.; Goldberg, K. Networked, cloud and fog robotics. In *Robotics Goes MOOC: Interaction*; Springer: Berlin/Heidelberg, Germany, 2025; pp. 271–313.
41. Zuo, Q.; Tao, D.; Qi, T.; Xie, J.; Zhou, Z.; Tian, Z.; Mingyu, Y. Industrial internet robot collaboration system and edge computing optimization. *arXiv* **2025**, arXiv:2504.02492. [[CrossRef](#)]
42. Ren, J.; Zhang, D.; He, S.; Zhang, Y.; Li, T. A survey on end-edge-cloud orchestrated network computing paradigms: Transparent computing, mobile edge computing, fog computing, and cloudlet. *ACM Comput. Surv. (CSUR)* **2019**, *52*, 1–36. [[CrossRef](#)]
43. Dey, S.; Mukherjee, A. Robotic slam: A review from fog computing and mobile edge computing perspective. In Proceedings of the Adjunct 13th International Conference on Mobile and Ubiquitous Systems: Computing Networking and Services, Hiroshima, Japan, 28 November–1 December 2016; pp. 153–158.
44. Dechouniotis, D.; Spatharakis, D.; Papavassiliou, S. Edge Robotics Experimentation over Next Generation IIoT Testbeds. In Proceedings of the NOMS 2022—2022 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 25–29 April 2022; pp. 1–3.
45. Li, W.; Fan, Y.; Jiang, A.; Liu, Z.; Pan, A.; Wei, B. Intelligent control method of live working robot based on cloud and edge computing terminal. In Proceedings of the 2020 10th IEEE International Conference on Power and Energy Systems (ICPES), Chengdu, China, 25–27 December 2020; pp. 321–325.
46. Wang, H. Collaborative task offloading strategy of UAV cluster using improved genetic algorithm in mobile edge computing. *J. Robot.* **2021**, *2021*, 3965689. [[CrossRef](#)]
47. Tahir, N.; Parasuraman, R. Utility ai for dynamic task offloading in the multi-edge infrastructure. In Proceedings of the 2023 Seventh IEEE International Conference on Robotic Computing (IRC), Laguna Hills, CA, USA, 11–13 December 2023; pp. 331–338.
48. Tahir, N.; Parasuraman, R. Consensus-based resource scheduling for collaborative multi-robot tasks. In Proceedings of the 2023 Seventh IEEE International Conference on Robotic Computing (IRC), Laguna Hills, CA, USA, 11–13 December 2023; pp. 323–330.
49. Baruffa, G.; Detti, A.; Rugini, L.; Crocetti, F.; Banelli, P.; Costante, G.; Valigi, P. Ai-driven ground robots: Mobile edge computing and mmwave communications at work. *IEEE Open J. Commun. Soc.* **2024**, *5*, 3104–3119. [[CrossRef](#)]
50. Klaas, T.; Lambrecht, J.; Funk, E. Semantic Local Planning for Mobile Robots through Path Optimization Services on the Edge: A Scenario-based Evaluation. In Proceedings of the 2020 25th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA), Vienna, Austria, 8–11 September 2020; pp. 711–718.
51. Lambrecht, J.; Funk, E. Edge-Enabled Autonomous Navigation and Computer Vision as a Service: A Study on Mobile Robot's Onboard Energy Consumption and Computing Requirements. In Proceedings of the ROBOT, Porto, Portugal, 20–22 November 2019.
52. Antevski, K. Enhancing Edge robotics through the use of context information. In Proceedings of the Workshop on Experimentation and Measurements in 5G, Heraklion, Greece, 4 December 2018; pp. 7–12.
53. Groshev, M.; Martín-Pérez, J.; Antevski, K.; De La Oliva, A.; Bernardos, C.J. COTORRA: Context-Aware testbed for robotic applications. In Proceedings of the MobileServerless 2021, 2021 1st Workshop on Serverless Mobile Networking for 6G Communications, Virtual, 25 June 2021; pp. 7–12. [[CrossRef](#)]
54. Zeng, Q.; Wang, Z.; Zhou, Y.; Wu, H.; Yang, L.; Huang, K. Knowledge-based ultra-low-latency semantic communications for robotic edge intelligence. *IEEE Trans. Commun.* **2024**, *early access*.
55. Thong Tran, T.; Zhang, Y.C.; Liao, W.T.; Lin, Y.J.; Li, M.C.; Huang, H.S. An autonomous mobile robot system based on serverless computing and edge computing. In Proceedings of the APNOMS 2020—2020 21st Asia-Pacific Network Operations and Management Symposium: Towards Service and Networking Intelligence for Humanity, Daegu, Republic of Korea, 22–25 September 2020; pp. 334–337. [[CrossRef](#)]

56. Huang, P.; Zeng, L.; Luo, K.; Guo, J.; Zhou, Z.; Chen, X. ColaSLAM: Real-time multi-robot collaborative laser SLAM via edge computing. In Proceedings of the IEEE/CIC International Conference on Communications in China, ICCIC 2021, Xiamen, China, 28–30 July 2021; pp. 242–247. [\[CrossRef\]](#)
57. Huang, P.; Zeng, L.; Chen, X.; Luo, K.; Zhou, Z.; Yu, S. Edge robotics: Edge-computing-accelerated multirobot simultaneous localization and mapping. *IEEE Internet Things J.* **2022**, *9*, 14087–14102. [\[CrossRef\]](#)
58. Liu, X.; Wen, S.; Zhao, J.; Qiu, T.Z.; Zhang, H. Edge-assisted multi-robot visual-inertial slam with efficient communication. *IEEE Trans. Autom. Sci. Eng.* **2025**, *22*, 2186–2198. [\[CrossRef\]](#)
59. Sarker, V.K.; Pe, J. Offloading SLAM for Indoor Mobile Robots with Edge-Fog-Cloud Computing. In Proceedings of the 2019 1st International Conference on Advances in Science, Engineering and Robotics Technology (ICASERT), Dhaka, Bangladesh, 3–5 May 2019.
60. Cui, X.; Lu, C.; Wang, J. 3D Semantic Map Construction Using Improved ORB-SLAM2 for Mobile Robot in Edge Computing Environment. *IEEE Access* **2020**, *8*, 67179–67191. [\[CrossRef\]](#)
61. Ben Ali, A.J.; Kouroshli, M.; Semenova, S.; Hashemifar, Z.S.; Ko, S.Y.; Dantu, K. Edge-slam: Edge-assisted visual simultaneous localization and mapping. *ACM Trans. Embed. Comput. Syst.* **2022**, *22*, 18. [\[CrossRef\]](#)
62. Chen, Q.; Zhu, H.; Yang, L.; Chen, X.; Pollin, S.; Vinogradov, E. Edge computing assisted autonomous flight for UAV: Synergies between vision and communications. *IEEE Commun. Mag.* **2021**, *59*, 28–33. [\[CrossRef\]](#)
63. Palani, U.; Engineering, C.; Sadhana, R.; Engineering, C.; Mahesh, S.S.; Analyst, P.; Vasanthi, D.; Engineering, C. Edge Computing Based Autonomous Robot for Secured Industrial IoT. In Proceedings of the 2022 IEEE International Conference on Data Science and Information System (ICDSIS), Hassan, India, 29–30 July 2022.
64. Fan, Z.; Chen, W.; Zhu, G.; You, Y.; Deng, F.; Hou, Y.; Liang, W.; Fu, R.; Xin, J.; Chen, J.; et al. Collaborative Robot Transport System Based on Edge Computing. In Proceedings of the 9th IEEE International Conference on Cyber Technology in Automation, Control and Intelligent Systems, CYBER 2019, Suzhou, China, 29 July–2 August 2019; pp. 1320–1326. [\[CrossRef\]](#)
65. Li, J.; Yin, J.; Deng, L. A robot vision navigation method using deep learning in edge computing environment. *Eurasip J. Adv. Signal Process.* **2021**, *2021*, 22. [\[CrossRef\]](#)
66. Qingqing, L.; Queralta, J.P.; Gia, T.N.; Westerlund, T. Offloading monocular visual odometry with edge computing: Optimizing image quality in multi-robot systems. In Proceedings of the 2019 5th International Conference on Systems, Control and Communications, Wuhan, China, 21–23 December 2019; pp. 22–26. [\[CrossRef\]](#)
67. Hayat, S.; Jung, R.; Hellwagner, H.; Bettstetter, C.; Emini, D.; Schnieders, D. Edge Computing in 5G for Drone Navigation: What to Offload? *IEEE Robot. Autom. Lett.* **2021**, *6*, 2571–2578. [\[CrossRef\]](#)
68. Li, G.; Han, R.; Wang, S.; Gao, F.; Eldar, Y.C.; Xu, C. Edge accelerated robot navigation with collaborative motion planning. *IEEE/ASME Trans. Mechatron.* **2024**, *30*, 1166–1178. [\[CrossRef\]](#)
69. Asavasirikulkij, C.; Mathong, C.; Sinthumongkolchai, T.; Chanchaen, R.; Asdomwised, W. Low Latency Peer to Peer Robot Wireless Communication with Edge Computing. In Proceedings of the 2021 IEEE 11th International Conference on System Engineering and Technology, ICSET 2021, Shah Alam, Malaysia, 6 November 2021; pp. 100–105. [\[CrossRef\]](#)
70. Yin, R.; Shen, Y.; Zhu, H.; Chen, X.; Wu, C. Time-critical tasks implementation in MEC based multi-robot cooperation systems. *China Commun.* **2022**, *19*, 199–215. [\[CrossRef\]](#)
71. Tahir, N.; Parasuraman, R.; Sun, H. Communication-aware consistent edge selection for mobile users and autonomous vehicles. In Proceedings of the 2024 IEEE 100th Vehicular Technology Conference (VTC2024-Fall), Washington, DC, USA, 7–10 October 2024; pp. 1–7.
72. Chen, K.; Wang, M.; Gualtieri, M.; Tian, N.; Juetter, C.; Ren, L.; Ichnowski, J.; Kubiatoicz, J.; Goldberg, K. Fogros2-ls: A location-independent fog robotics framework for latency sensitive ros2 applications. In Proceedings of the 2024 IEEE International Conference on Robotics and Automation (ICRA), Yokohama, Japan, 13–17 May 2024; pp. 10581–10587.
73. Qingqing, L.; Yuhong, F.; Pe, J. Edge Computing for Mobile Robots: Multi-Robot Feature-Based Lidar Odometry with FPGAs. In Proceedings of the 2019 Twelfth International Conference on Mobile Computing and Ubiquitous Network (ICMU), Kathmandu, Nepal, 4–6 November 2019; pp. 4–5.
74. Huang, X.; Yu, R.; Ye, D.; Shu, L.; Xie, S. Efficient Workload Allocation and User-Centric Utility Maximization for Task Scheduling in Collaborative Vehicular Edge Computing. *IEEE Trans. Veh. Technol.* **2021**, *70*, 3773–3787. [\[CrossRef\]](#)
75. Bouhoula, S.; Avgeris, M.; Leivadreas, A.; Lambadaris, I. Drl-based trajectory planning and sensor task scheduling for edge robotics. In Proceedings of the 2024 IEEE 10th World Forum on Internet of Things (WF-IoT), Ottawa, ON, Canada, 10–13 November 2024; pp. 352–357.
76. Wang, X.; Guo, H. Mobility-Aware Computation Offloading for Swarm Robotics using Deep Reinforcement Learning. In Proceedings of the 2021 IEEE 18th Annual Consumer Communications & Networking Conference (CCNC), Las Vegas, NV, USA, 9–12 January 2021; pp. 1–4. [\[CrossRef\]](#)
77. Farahbakhsh, F.; Shahidinejad, A.; Ghobaei-Arani, M. Multiuser context-aware computation offloading in mobile edge computing based on Bayesian learning automata. *Trans. Emerg. Telecommun. Technol.* **2021**, *32*, e4127. [\[CrossRef\]](#)

78. Zeng, L.; Ye, S.; Chen, X.; Yang, Y. Implementation of big ai models for wireless networks with collaborative edge computing. *IEEE Wirel. Commun.* **2024**, *31*, 50–58. [[CrossRef](#)]
79. Jahan, F.; Sun, W.; Niyaz, Q.; Alam, M. Security Modeling of Autonomous Systems: A Survey. *ACM Comput. Surv.* **2019**, *52*, 91. [[CrossRef](#)]
80. Ghosh, S.; Patro, R.; Raina, M.; Thejaswi, C.; Ganapathy, V. Secure group communication in wireless sensor networks. In Proceedings of the 2006 1st International Symposium on Wireless Pervasive Computing, Phuket, Thailand, 16–18 January 2006; p. 5. [[CrossRef](#)]
81. Chaâri, R.; Cheikhrouhou, O.; Koubâa, A.; Youssef, H. Dynamic computation offloading for ground and flying robots: Taxonomy, state of art, and future directions. *Comput. Sci. Rev.* **2022**, *45*, 100488. [[CrossRef](#)]

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.